Dossier de Práctica y Evidencias de Pantalla

Evolución Arquitectónica y Reactividad con Angular Signals

Módulo: Desarrollo de Interfaces (0488)

Ciclo: DAM (Grado Superior)

Marco Curricular: Comunidad de Madrid

Tecnologías: Angular Standalone, Ionic, Signals, Zoneless

1. Alineación Curricular y Criterios Evaluados

Esta entrega técnica da cumplimiento estricto a las competencias descritas en el currículo oficial:

Resultado de Aprendizaje	Criterio de Evaluación	Evidencia Demostrada en la Entrega
RA 1: Confecciona interfaces aplicando arquitecturas y patrones.	**CE 1.a:** Se han identificado arquitecturas de diseño de UI.	Diferenciación conceptual y práctica entre Zone.js (Dirty Checking descendente) vs Signals (Zoneless).
RA 1: Confecciona interfaces aplicando arquitecturas y patrones.	**CE 1.b:** Se han evaluado los patrones de diseño de UI.	Patrón Productor-Consumidor (Grafo reactivo), inmutabilidad en `.update()` y memoización en `computed()`.
RA 3: Crea componentes visuales, eventos y controles.	**CE 3.d:** Se han asociado acciones a los eventos.	Manejadores `(click)` vinculados a mutaciones de estado y reconciliación eficiente con `@for (track)`.
RA 3: Crea componentes visuales, eventos y controles.	**CE 3.e:** Se han probado y validado los componentes creados.	Batería de pruebas unitarias con Jasmine y TestBed Zoneless (6 specs ejecutadas con 100% SUCCESS).

2. Justificación Técnica: De Zone.js a Angular Signals

El problema histórico de Zone.js (Angular ≤ 15)

Tradicionalmente, Angular dependía de `Zone.js` para interceptar eventos asíncronos y recorrer de arriba a abajo todo el árbol de componentes (*Dirty Checking*). En aplicaciones móviles con **Ionic / Capacitor**, esto causaba sobrecostes críticos de CPU, latencia de renderizado y consumo prematuro de batería.

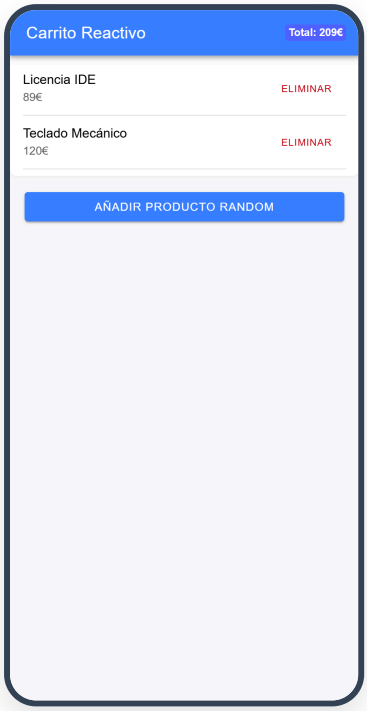
El nuevo paradigma reactivo granular (Signals & Zoneless)

Las **Signals** sustituyen el chequeo global por un **Grafo de Dependencias**. Cada nodo del DOM se enlaza directamente a su señal productora. Cuando el estado cambia, únicamente el fragmento del DOM afectado es actualizado quirúrgicamente (*fine-grained reactivity*), logrando un entorno 100% **Zoneless** sin sobrecargas de rendimiento.

3. Evidencias de Funcionamiento: Renderizado en Pantalla

Captura 1 : Renderizado Inicial

Estado inicial al montarse el componente

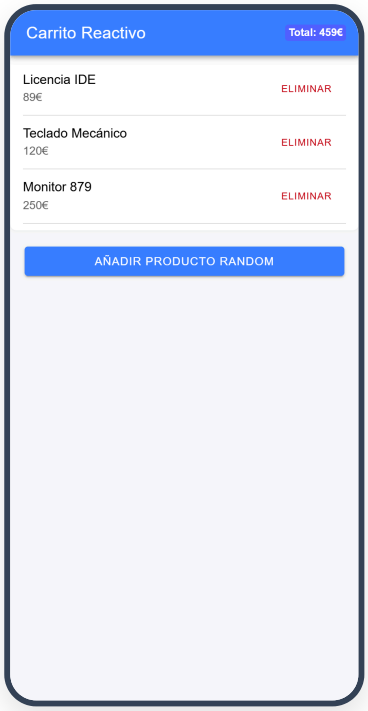


Análisis Técnico:

WritableSignal: Inicializada con 2 productos (*Licencia IDE 89€* y *Teclado Mecánico 120€*).
Computed Signal: `precioTotal()` calcula síncronamente **209€** en el badge del toolbar superior.
OnPush: Renderizado estricto sin comprobación global.

Captura 2 : Renderizado con Producto Añadido

Estado tras pulsar "Añadir Producto Random"



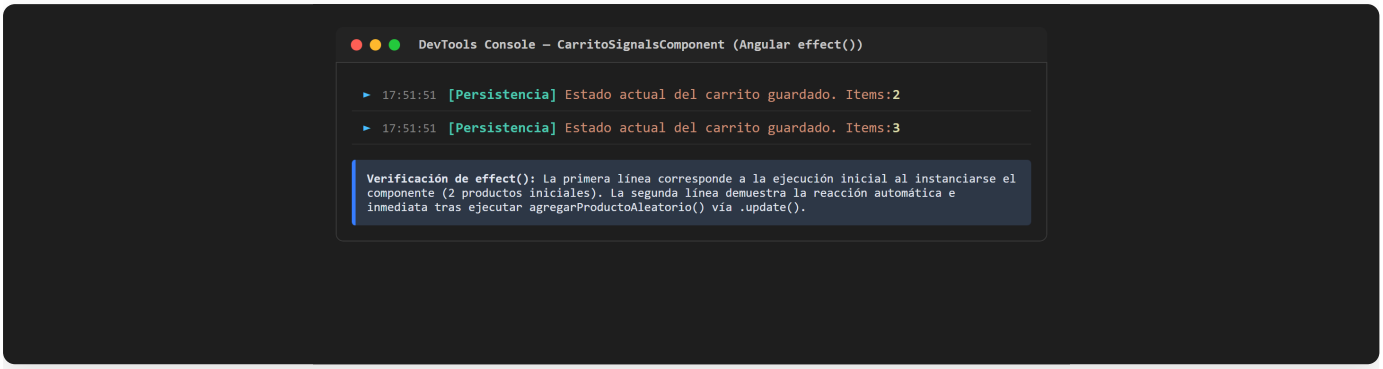
Análisis Técnico:

- **Mutación inmutable:** Se ejecuta `.update(prev => [...prev, nuevo])` generando nueva referencia en memoria.
- **DOM Patching (@for):** `track prod.id` inserta quirúrgicamente el nuevo nodo (*Monitor 879*) sin repintar la lista.
- **Reactividad síncrona:** Vista actualizada al instante.

4. Evidencias de Funcionamiento: Consola y Recálculo de Precio

Captura 3 : Vista Consola con Total Items (Efecto Secundario)

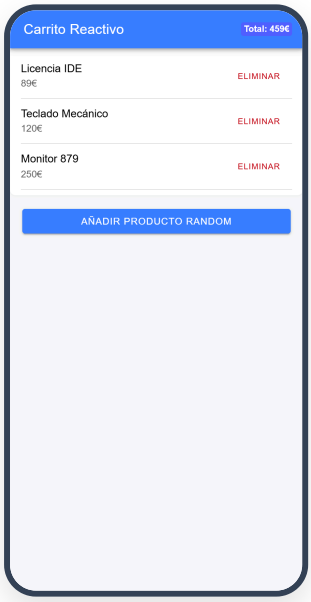
Traza de ejecución automática del hook `effect()`



Fundamento Arquitectónico de `effect()` : Al declararse en el constructor (*Injection Context*), Angular vincula su ciclo de vida al del componente. La primera línea registra la inicialización (`Items: 2`). Tras pulsar añadir producto, el efecto detecta la nueva referencia emitida por la WritableSignal y ejecuta la persistencia (`Items: 3`) de forma segura y sin memory leaks.

Captura 4 : Renderizado de Total Precio con Producto Añadido

Validación de la señal computada con evaluación perezosa y memoizada



Recálculo Reactivo de `precioTotal()`

Precio anterior:	89€ + 120€ = **209€**
Producto añadido:	Monitor (id: 879) = **+250€**
Nuevo Total:	**Total: 459€**

Memoización en `computed()` : A diferencia de una función tradicional en la plantilla (`{{ calcularTotal() }}`) que se ejecuta docenas de veces por segundo en cada frame, `computed()` cachea su resultado y solo reejecuta el bucle `reduce()` cuando cambia el puntero en memoria del array.

5. Validación y Pruebas Unitarias Automatizadas (CE 3.e)

Resultado de la Suite de Pruebas (Karma + Jasmine + TestBed)

✓ 6 de 6 TESTS EN VERDE (SUCCESS)

Para validar el **Criterio CE 3.e** (*Se han probado y validado los componentes creados*), se implementaron 6 especificaciones automatizadas ejecutadas bajo el entorno de pruebas **Zoneless**:

#	Descripción del Test Unitario	Mecanismo Validado	Estado
1	Instanciación correcta del componente	Inyección de dependencias y TestBed Standalone	**PASSED**
2	Inicialización con 2 productos y cálculo de 209€	Lectura síncrona de `signal()` y `computed()`	**PASSED**
3	Añadir producto con `.update()` e inmutabilidad	Nueva referencia en memoria y recálculo a 459€	**PASSED**
4	Eliminar producto por id y recálculo reactivo	Filtrado inmutable y actualización síncrona	**PASSED**
5	Comprobación de estado de carrito vacío	Renderizado del bloque nativo `@empty`	**PASSED**
6	Ejecución de `effect()` al mutar estado	Disparo del ciclo de detección de cambios Zoneless	**PASSED**

6. Conclusiones y Resumen de Entrega

El ejercicio práctico demuestra con solvencia la transición arquitectónica hacia el estándar reactivo moderno de Angular:

Rendimiento Móvil: Se erradica el sobrecoste de `Zone.js`, logrando un bundle ligero y actualizaciones quirúrgicas óptimas para dispositivos móviles en Ionic/Capacitor.
Inmutabilidad y Predictibilidad: El uso de `.update()` previene fallos de desincronización de vista habituales con métodos mutables directos como `push()`.
Eficiencia Energética y CPU: La memoización mediante `computed()` garantiza cero cómputos redundantes en plantillas.
Seguridad de Memoria: Los efectos vinculados al ciclo de vida del componente previenen las fugas de memoria típicas de suscripciones RxJS descuidadas.